Lab 1 - Bus Systems and Sensors

Temprature Sensor

Ahmed Mohammed - 2757098
Jamal Shamsan - 2756372
Het Wai Yan Tun -2757519

May 2,2026

Instructor: Prof. Dr. Rasmus Rettig

Lab Group 3
Information Engineering

Contents

1 Setup and Basic Operation

1.1 Task Summary

In this part, the DS18B20 temperature sensor was connected to an Arduino Nano on a breadboard using the 1-wire interface. The sensor requires three connections: 5V, GND, and the data line DQ (pin D10), with a 5 kΩ pull-up resistor between DQ and 5V. The provided Arduino sketch `TemperaturSensor.ino` was compiled and uploaded, and the sensor output was verified using the Serial Monitor. Additionally, the Python script `01_Read_USB_CSV_TimeStamped.py` was used to record timestamped measurements on the laptop. The function was tested by touching the temperature sensor. When the sensor was touched, the measured temperature increased because heat from the hand was transferred to the metal sensor housing.

1.2 Experimental Setup

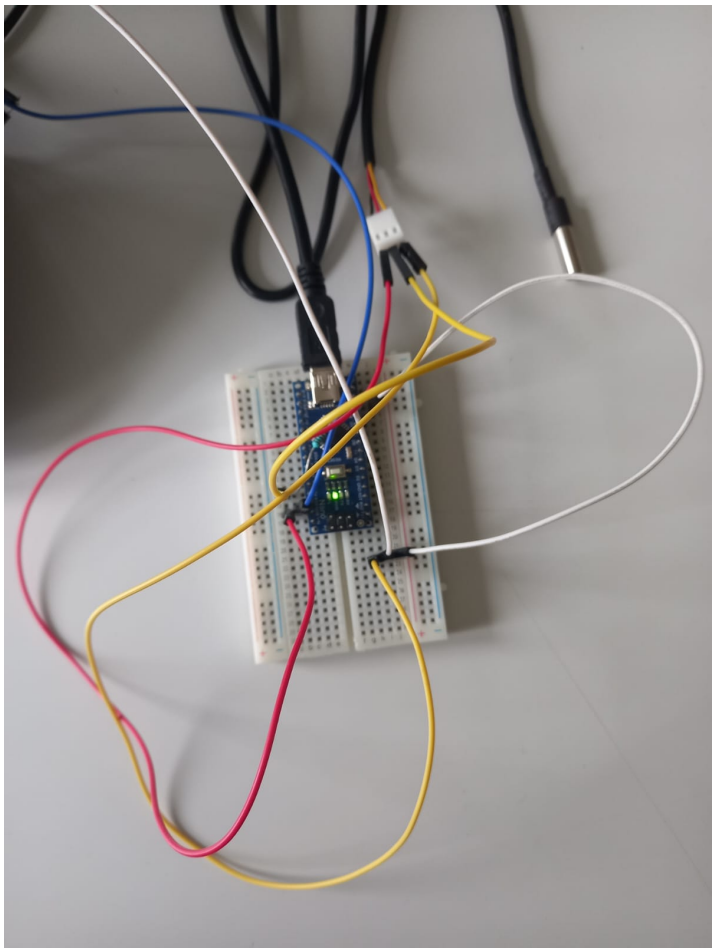

Figure 1: Hardware setup: Arduino Nano with DS18B20 sensor on breadboard.

1.3 Results

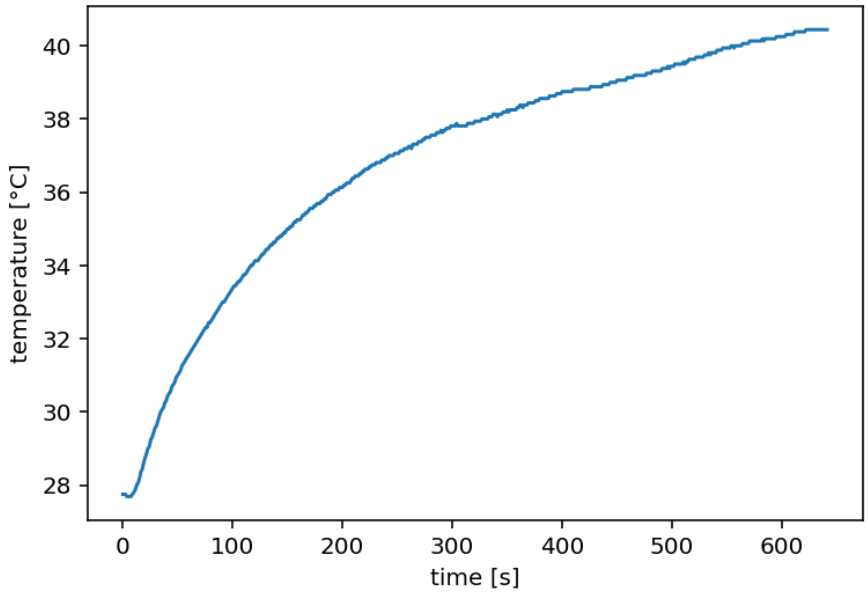

Figure 2: screenshot of the recorded measurement data.

The temperature-over-time plot shows that the sensor output increased during the measurement. At the beginning, the temperature was about 28 řC. After heating/touching the sensor, the temperature rose continuously up to about 40 řC. The curve rises faster at the beginning and then more slowly. This shows that the DS18B20 temperature sensor reacted correctly to the temperature change. It also confirms that the measured values were successfully sent from the Arduino to the laptop and recorded by the Python program.

1.4 Hardware Block Diagram

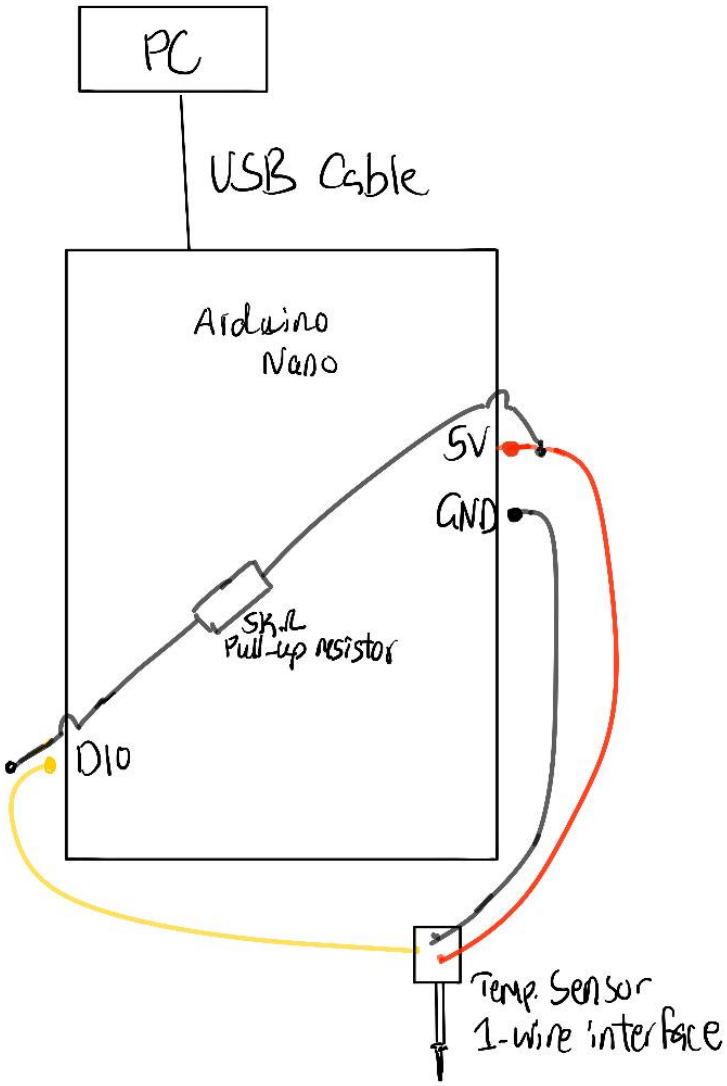

Figure 3: Hardware Block Diagram

1.5 Data Flow Diagram and Program Analysis

Arduino program analysis The Arduino program reads the temperature from the DS18B20 sensor. The sensor is connected to the Arduino by the 1-wire data line on pin D10.

At the beginning, the Arduino starts the serial connection with a baud rate of 19200. This connection is used to send the measured temperature values to the laptop.

The measurement is repeated again and again in the loop() function. In each cycle, the Arduino asks the sensor to measure the temperature. Then it waits shortly until the sensor has finished the measurement. After that, the Arduino reads the sensor data, converts it into a temperature value in degrees Celsius, and sends this value to the laptop using the USB serial connection.

So, the Arduino has two main tasks:

1. Read the temperature from the sensor.
2. Send the temperature value to the laptop.

The Python program `01_Read_USB_CSV_TimeStamped.py` runs on the laptop and receives the temperature values from the Arduino through the USB serial connection.

It opens the serial port with the same baud rate as the Arduino. This is important so that both devices can communicate correctly.

The program reads one temperature value after another, converts the received text into a number, and adds a timestamp. This timestamp shows how many seconds have passed since the Python program started.

Then the program saves the timestamp and the temperature value into a CSV file called `testdata.csv`.

At the end, the program also creates a temperature-over-time plot and a histogram of the measured values.

How the measurement results get to the laptop:

Temperature change
↓
DS18B20 temperature sensor
↓
1-wire data line DQ
↓
Arduino Nano, pin D10
↓
Arduino converts raw sensor data to °C
↓
`Serial.println(celsius) -- Sending the value to the laptop`
↓
USB serial connection
↓
Laptop serial port COM27
↓
Python program reads the value
↓
CSV file: `testdata.csv`

How the timestamp is generated The timestamp is created in the Python program with the line `timestamp = time.time() - start_zeit`.

Here, `start_zeit` is the time when the program started. `time.time()` gives the current time.

By subtracting the start time from the current time, the program calculates how many seconds have passed since the measurement started.

This timestamp is then saved together with the temperature value in the CSV file.

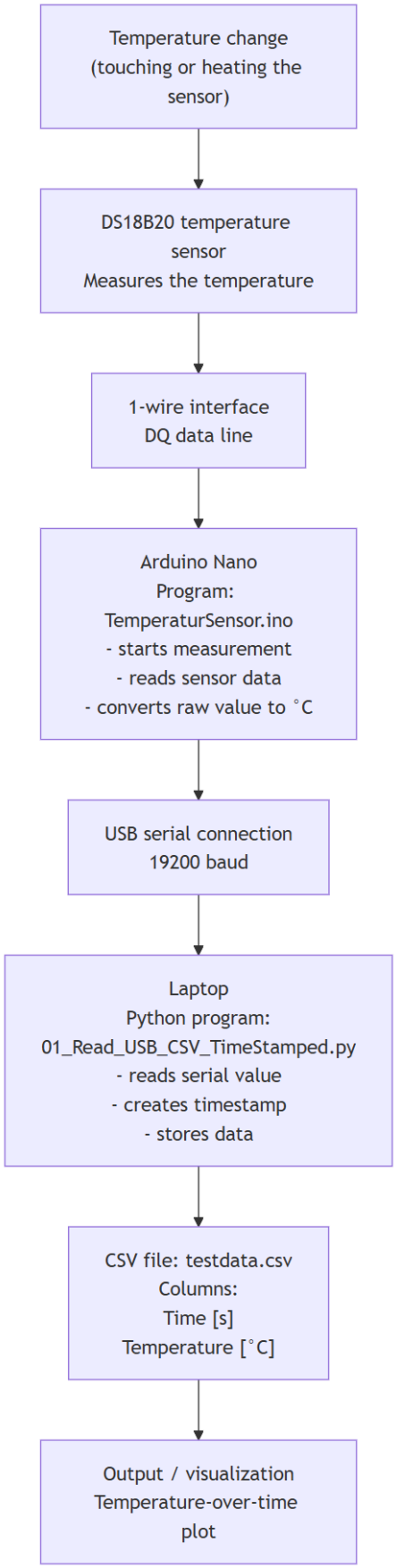

Figure 4: Data flow diagram

2 Investigation of the 1-Wire Interface

2.1 Task Summary

In this part, the 1-wire communication signal on the DQ pin was examined using an oscilloscope. The probe was connected to pin D10, and the voltage was measured relative to GND across multiple time scales. The goal was to identify the structure of the transmitted telegrams and decode the first bytes of a communication frame to determine which commands are sent by the Arduino and which are sent by the sensor in response.

2.2 Experimental Setup

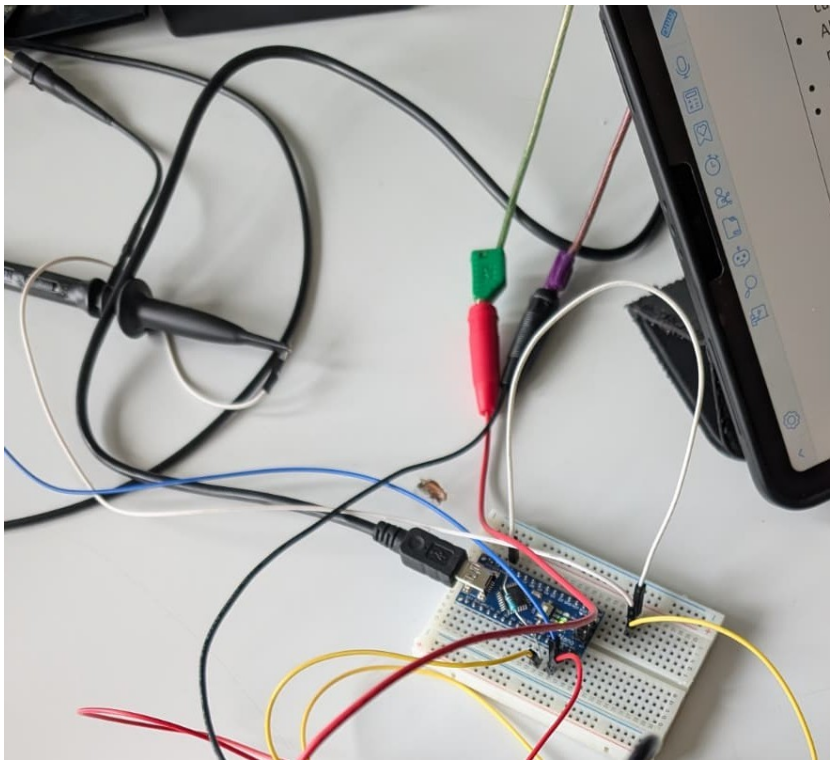

Figure 5: Oscilloscope probe connected to DQ pin of the DS18B20 sensor.

The first wire is the white wire, connected to D10 on one side and to the oscilloscope probe on the other. Moreover, the other wire is the blue wire, from one side to the Arduino ground, and the other is to the oscilloscope ground probe. The probe wire is connected to Channel 1 on the oscilloscope.

2.3 Oscilloscope Measurements

The following figures show the DQ signal at five different timescales.

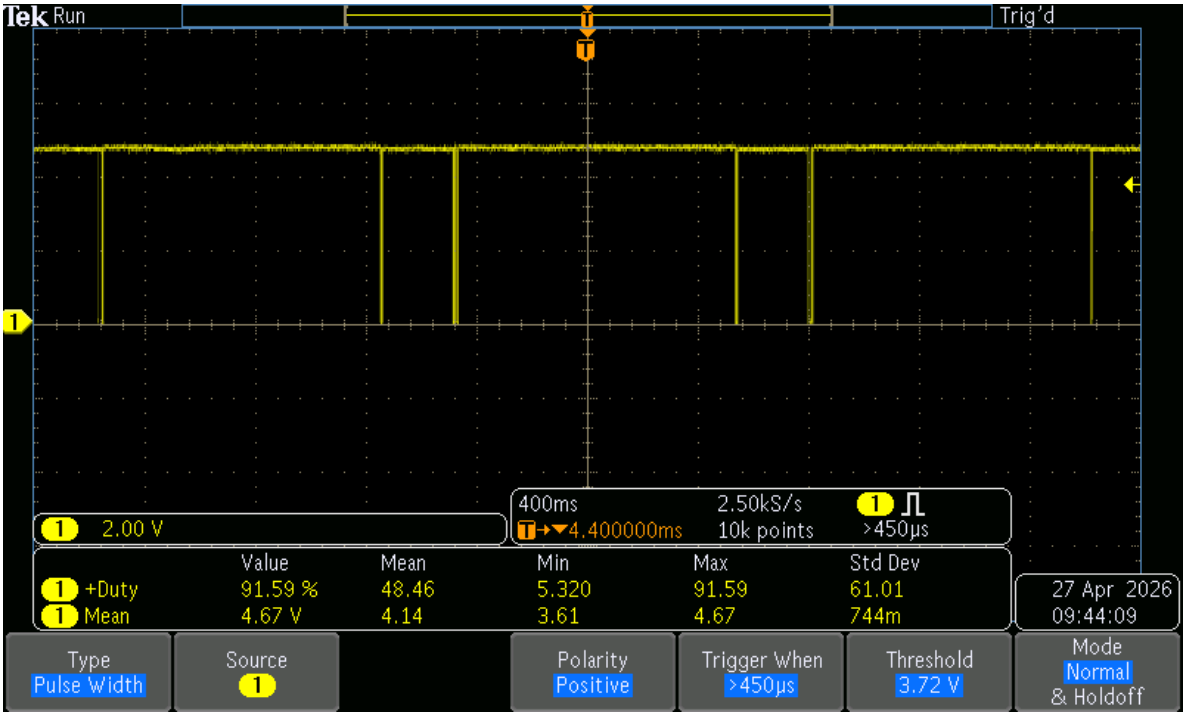

Figure 6: 400 ms/div

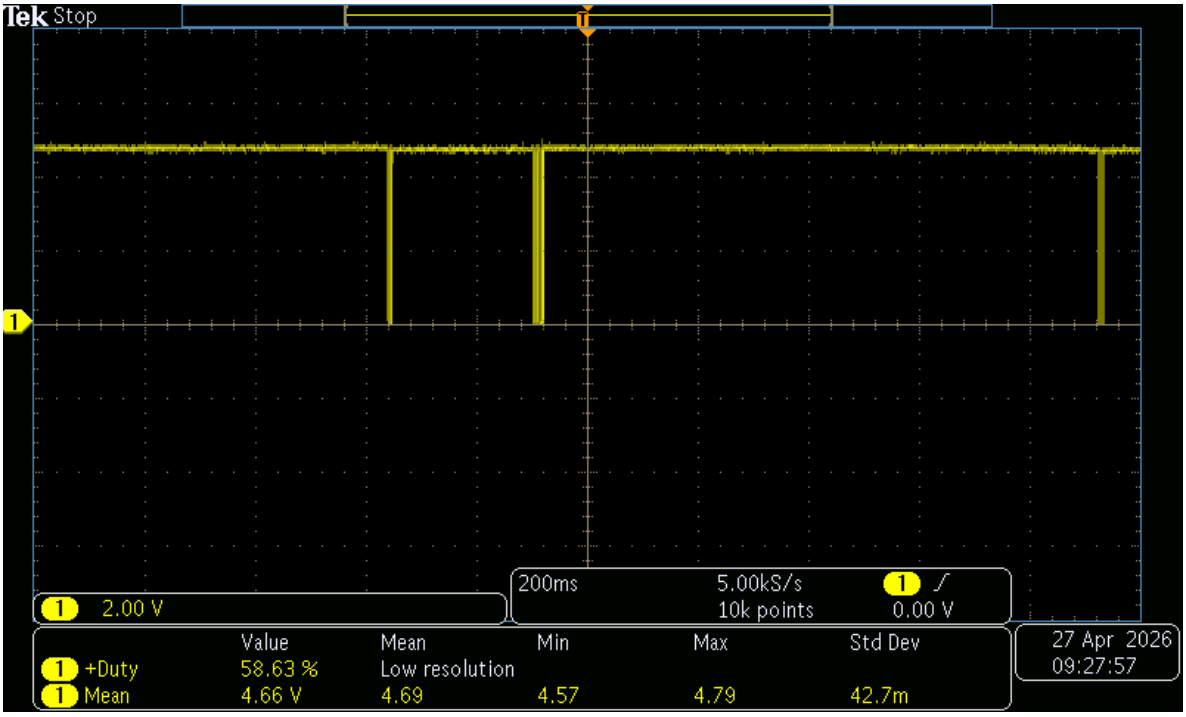

Figure 7: 200 ms/div

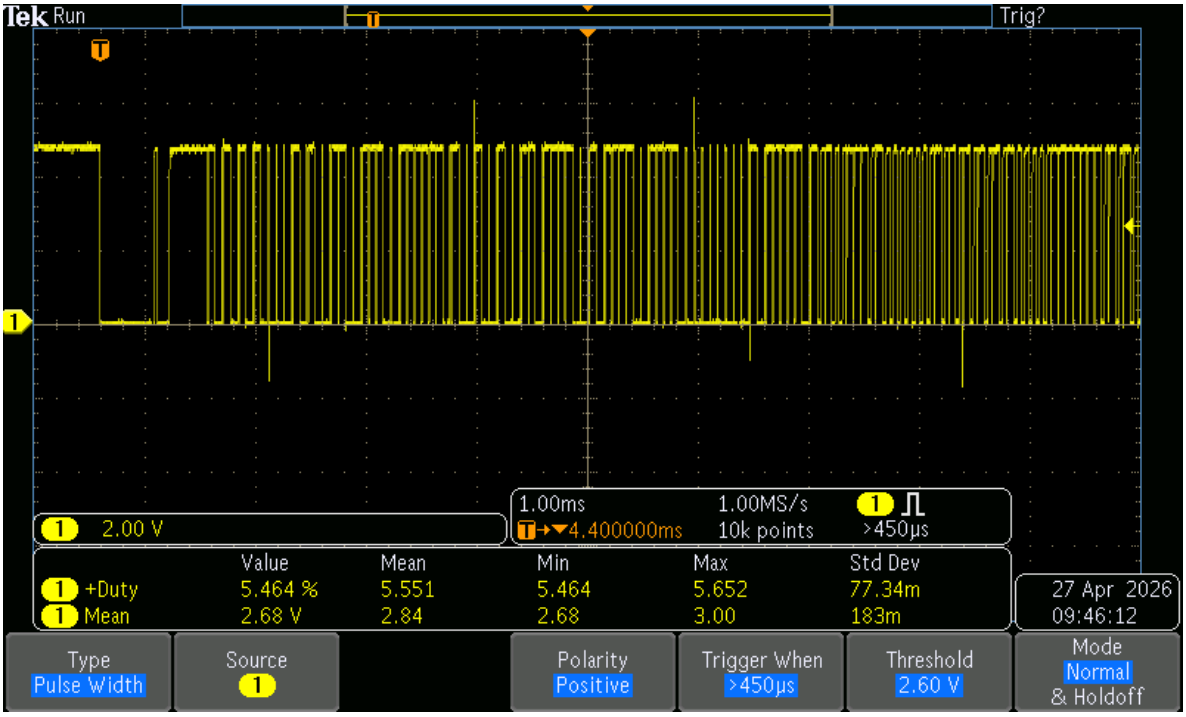

Figure 8: 1 ms/div

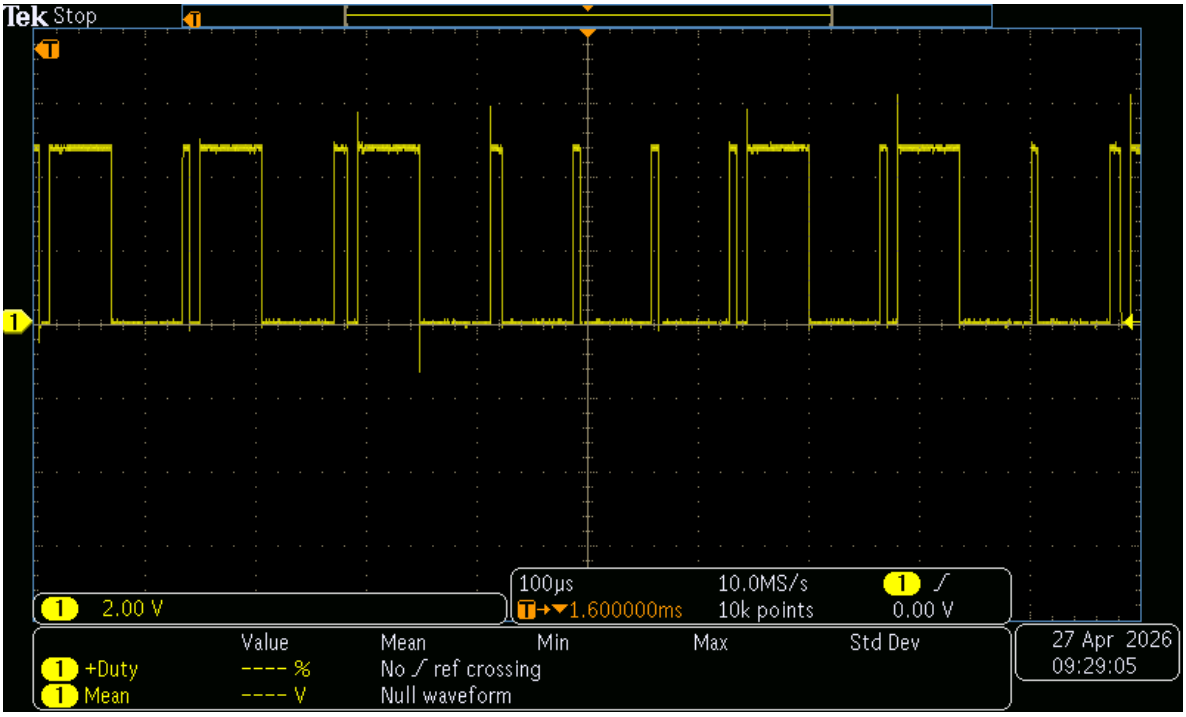

Figure 9: 100 μs/div

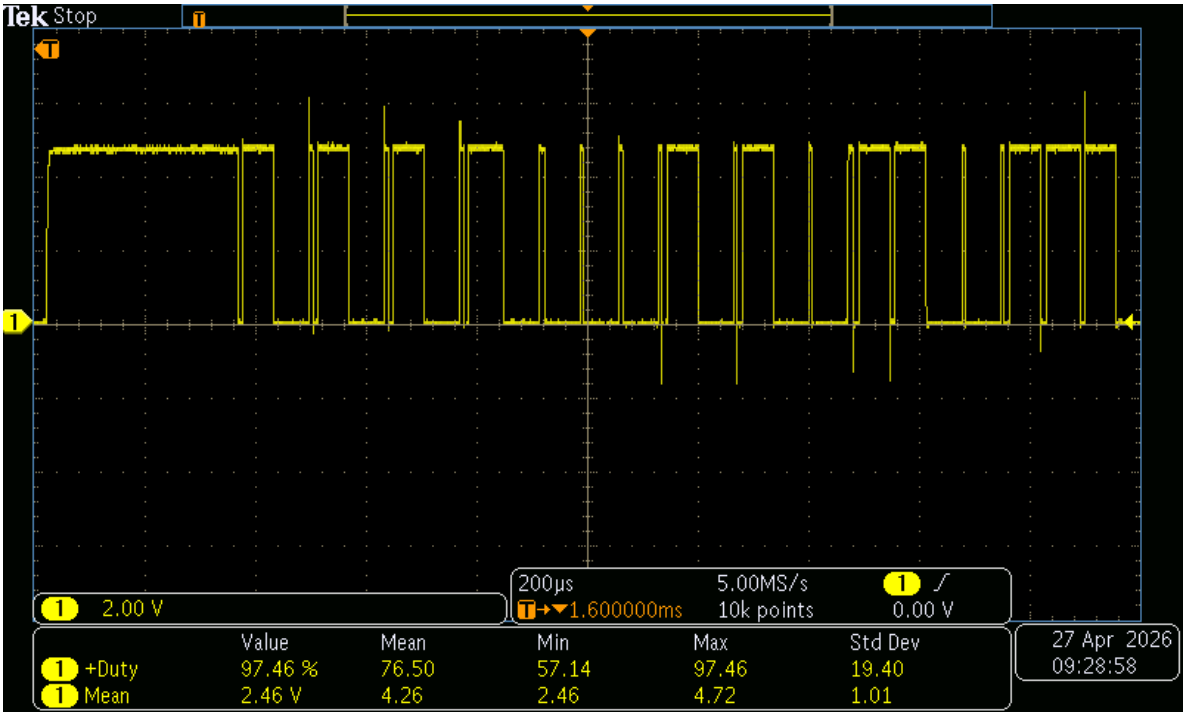

Figure 10: 200 μs/div

2.4 Annotated Telegram

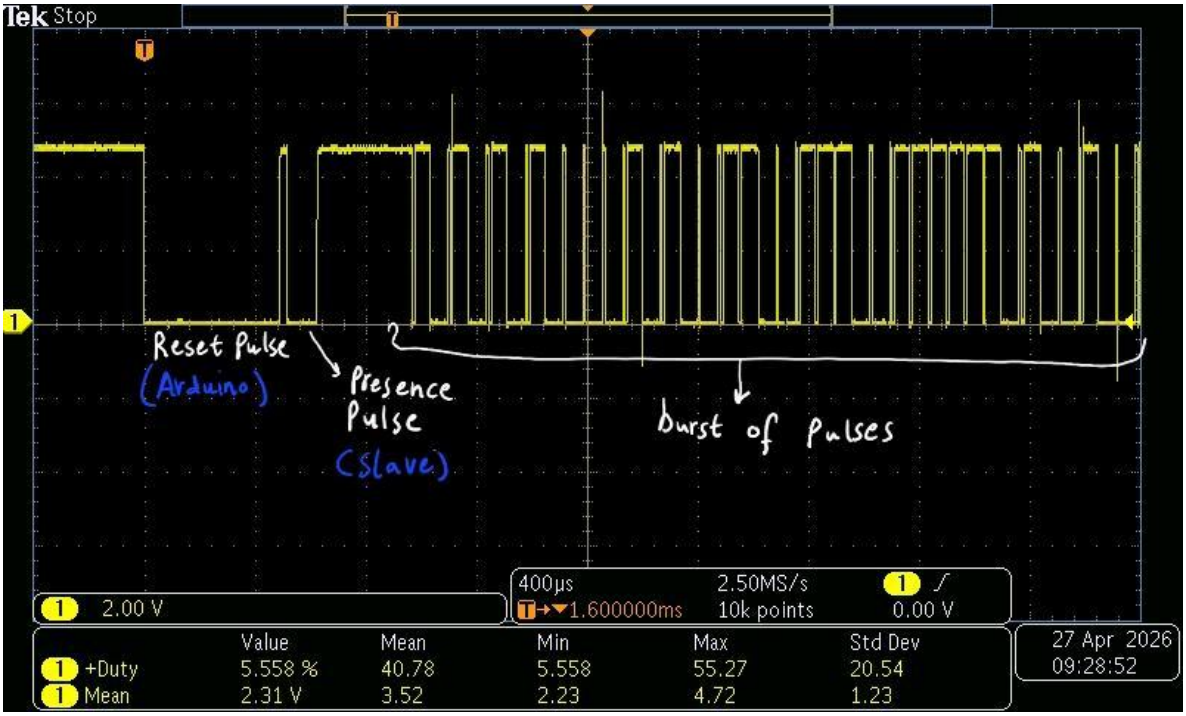

Figure 11: Annotated 1-wire telegram. The first 5 bytes after the Start of Frame are labelled.

Figure 11 shows the DQ signal captured at 400 µs/div, covering approximately 4 ms of the communication. The waveform begins with a long LOW pulse of approximately 480 µs, which is the Reset pulse generated by the Arduino master to initiate communication. Immediately following, a short LOW dip of approximately 60–240 µs is visible this is the Presence pulse sent by the DS18B20 sensor to confirm it is connected and ready.

After the presence pulse, the data transmission begins. In the 1-wire protocol, each bit occupies a fixed 60 µs time slot. A narrow LOW pulse, where the line is pulled LOW briefly and returns HIGH quickly due to the pull-up resistor, encodes a logic 1. A wide LOW pulse, where the line remains LOW for the full 60 µs slot, encodes a logic 0. Data is transmitted LSB first [1].

Decoding the visible bit slots after the presence pulse yields the following bytes:

- **Byte 1: `0xCC` (binary: 00110011) Skip ROM.** This command instructs the sensor to skip the 64-bit ROM identification step, which is valid when only one sensor is present on the bus.
- **Byte 2: `0x44` (binary: 00100010) Convert T.** This command triggers the DS18B20 to perform an internal temperature measurement and store the result in its scratchpad memory.

The wide LOW pulse, after the burst of pulses, marks the beginning of a new Reset pulse, indicating that the Arduino is initiating a second telegram to read back the converted temperature value.

2.5 Transmission Frequency

To determine how often a new temperature value is transferred to the bus, the DQ signal was examined at two wider timescales: 400 ms/div and 200 ms/div, as shown in Figures 6 and 7 respectively.

At 400 ms/div, the total visible time window spans 4000 ms across 10 divisions. Approximately four Reset pulses are visible within this window, yielding a period of approximately 1000 ms between consecutive telegrams. This is confirmed by the 200 ms/div measurement, where two Reset pulses are clearly visible within the 2000 ms window, again giving a period of approximately 1000 ms.

This period is consistent with the DS18B20 datasheet, which specifies a maximum conversion time of 750 ms at the default 12-bit resolution. The remaining ≈250 ms is used for the Read Scratchpad telegram and the inter-telegram idle time. Therefore, a **new temperature value is transferred to the bus approximately once per second**.

3 Determination of the Time Behaviour T(t)

3.1 Task Summary

In this part, the thermal response of the DS18B20 sensor to a step change in Peltier current was measured. Starting from room temperature with the Peltier at 0 A, the current was abruptly switched to 0.5 A and the resulting temperature rise was recorded as a time series. The data was then fitted to an exponential model to extract the characteristic time constant τ.

3.2 Experimental Setup

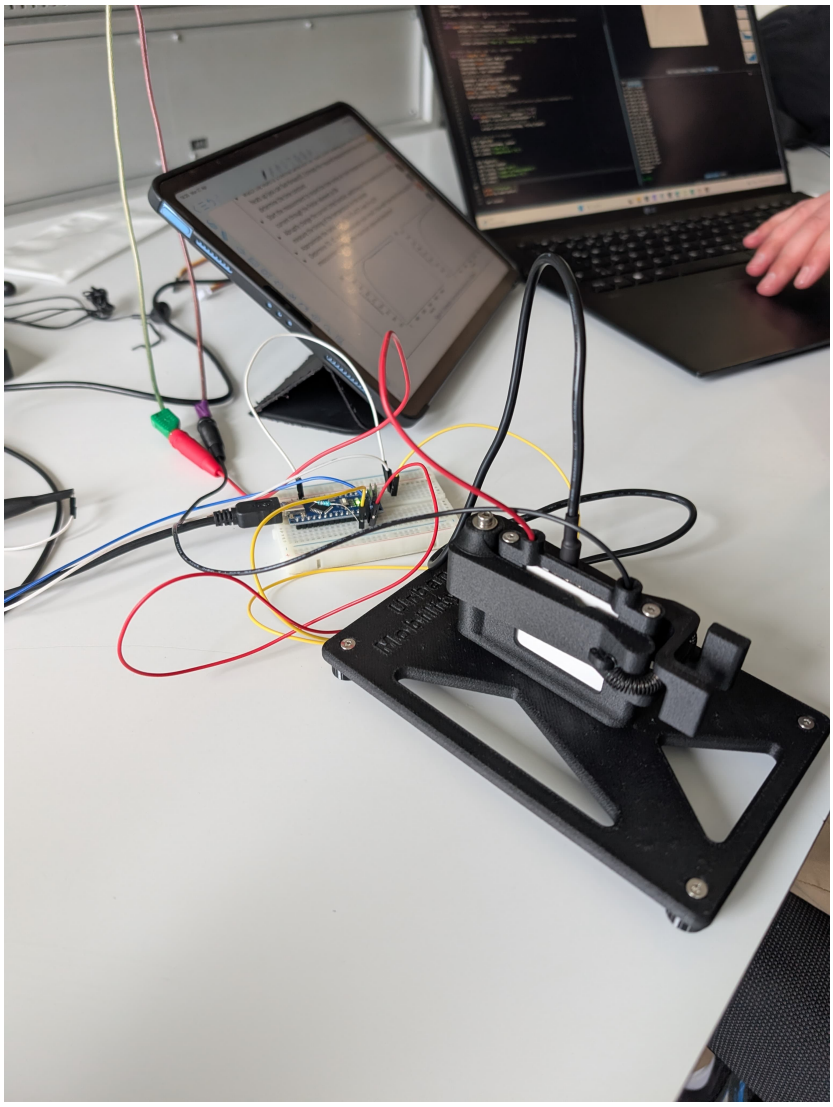

Figure 12: DS18B20 sensor mounted on a Peltier element and connected to an Arduino.

3.3 Measured Time Series

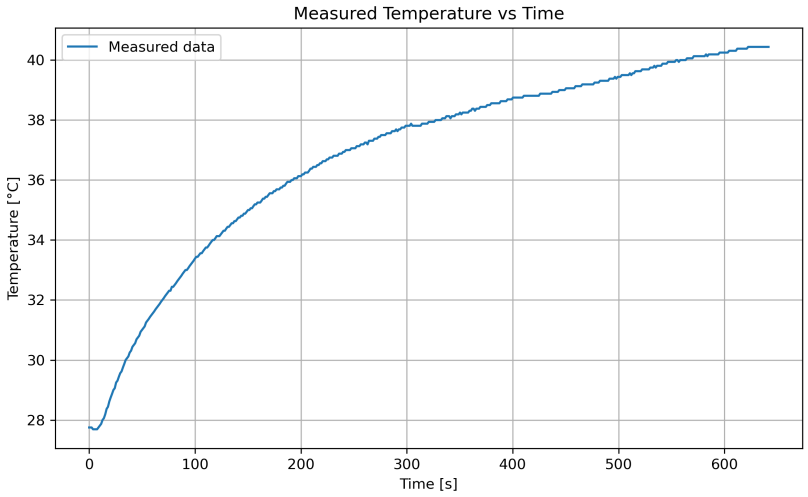

Figure 13: Measured temperature as a function of time for $I_{\text{Peltier}} = 0.5\,\text{A}$. The Peltier was switched on at $t_0 \approx 10$ s.

3.4 Curve Fitting and Parameter Determination

The measured data was approximated by the following model:

$$T(t) = T_0 + \Delta T \left(1 - e^{-(t-t_0)/\tau}\right) \tag{1}$$

The fitted parameters are summarised in Table 1.

Table 1: Fitted parameters of the exponential model (Equation 1).

Parameter	**Value**	**Description**
T_0	27.75 °C	Initial temperature at $t < t_0$
t_0	10 s	Time of current step
τ	170.8 s	Thermal time constant
ΔT	12.69 °C	Total temperature rise

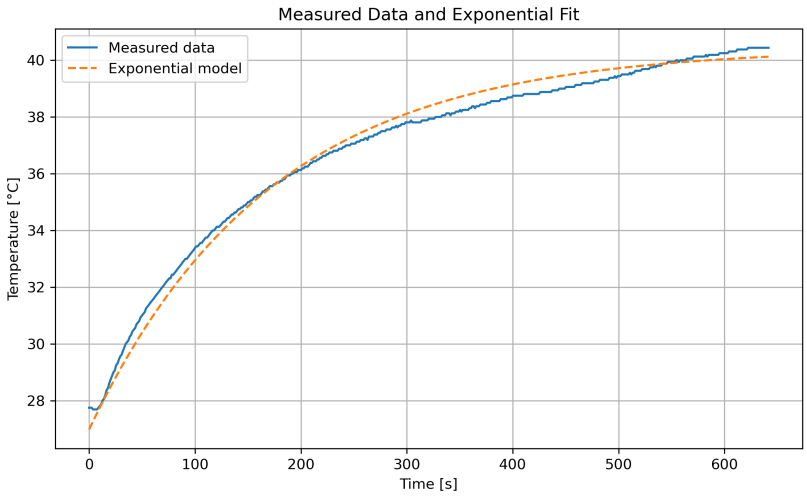

Figure 14: Fit of Equation 1 to the measured time series.

3.5 Discussion of Errors

The fitted curve matches the measured data well in the intermediate phase of the heating process, while larger deviations occur at the beginning and near the steady-state temperature. process. The maximum observed deviation between measurement and model is $\Delta T_{\max} = 0.77\,^{\circ}\mathrm{C}$, occurring at $t \approx 0\,\mathrm{s}$.This is due to the initial transient behaviour and non-ideal switching conditions, which are not fully captured by the exponential model.

Possible sources of error include thermal contact resistance between the sensor and the Peltier surface, heat dissipation to the surrounding air, and the discrete resolution of the DS18B20 (0.0625 °C in 12-bit mode). Additionally, the exponential model assumes a first-order thermal system, which may not fully capture the real behaviour of the setup.

References

[1] Analog Devices, *DS18B20 Programmable Resolution 1-Wire Digital Thermometer*, Accessed: 2026-04-27, 2019. [Online]. Available: https://www.analog.com/media/en/technical-documentation/data-sheets/ds18b20.pdf.

[2] Wikipedia contributors, *1-Wire — Bus System*, Accessed: 2026-04-27, 2024. [Online]. Available: https://de.m.wikipedia.org/wiki/1-Wire.

[3] R. Rettig, *Lecture 4: Bussystems and Sensors*, Lecture slides, 29.4.26, 2026.